

Despliegue del Aplicativo en 3 Fases

Sistema ReservaEspacios — ASP.NET Core (.NET 10) y MySQL

Instalación manual · Docker y Docker Compose · Vagrant con Puppet

Proyecto:	ReservaEspacios (sistema de reserva de espacios)
Alumno(a):	Luis Eder Aguilar Martinez
Materia:	Procesos de Desarrollo de Software
Docente:	Lopez Herrera Juan Luis
Fecha de entrega:	12/06/2026

Índice

Índice	2
Introducción	3
Características del sistema	3
Herramientas de despliegue	3
Fase 1. Instalación manual del servidor Linux	4
1.1 Creación de la máquina virtual e instalación de Ubuntu Server	4
1.2 Configuración inicial del servidor	6
1.3 Instalación y configuración de la base de datos (MySQL)	7
1.4 Instalación del SDK de .NET 10	9
1.5 Despliegue del sistema y creación del esquema	10
1.6 Ejecución y verificación	12
Fase 2. Despliegue con Docker y Docker Compose	14
2.1 Instalación de Docker y Docker Compose	14
2.2 Estructura del proyecto	16
2.3 Generación del esquema de la base de datos	17
2.4 Dockerfile del sistema	18
2.5 Archivo docker-compose.yml	20
2.6 Construcción, levantamiento y verificación	22
Fase 3. Servidor automático con Vagrant y Puppet	25
3.1 Instalación de Vagrant en el equipo anfitrión	25
3.2 Estructura del proyecto	25
3.3 Vagrantfile utilizado	26
3.4 Manifiesto de Puppet	28
3.5 Creación automática del servidor	29
3.6 Verificación dentro del servidor	30
3.7 Verificación del sistema en el navegador	32
Resumen de evidencias por fase	33
Conclusión	33

Introducción

Este documento describe el despliegue del sistema ReservaEspacios junto con su base de datos, aplicando tres estrategias de implementación cada vez más automatizadas. La primera fase instala todo manualmente sobre un servidor Linux; la segunda encapsula el sistema y la base de datos en contenedores Docker orquestados con Docker Compose; y la tercera reproduce ese mismo despliegue sobre un servidor Linux creado automáticamente con Vagrant y aprovisionado con Puppet.

Las tres fases que se documentan son:

- Fase 1. Instalación manual de un servidor Linux con el sistema y la base de datos ejecutándose directamente sobre el servidor.
- Fase 2. El sistema en un contenedor Docker y la base de datos en otro, desplegados sobre el servidor Linux con Docker Compose.
- Fase 3. El mismo despliegue en contenedores, sobre un nuevo servidor Linux creado de forma automática con Vagrant y con las herramientas instaladas mediante Puppet.

Características del sistema

- Aplicación web desarrollada en ASP.NET Core con C#, sobre .NET 10.
- Acceso a datos con Entity Framework Core (AppDbContext) y proveedor Pomelo para MySQL.
- Base de datos MySQL llamada «reservas_fei».
- Esquema de base de datos administrado mediante migraciones de Entity Framework Core.
- Pruebas unitarias con xUnit y FluentAssertions (proyecto ReservaEspacios.Tests).

Herramientas de despliegue

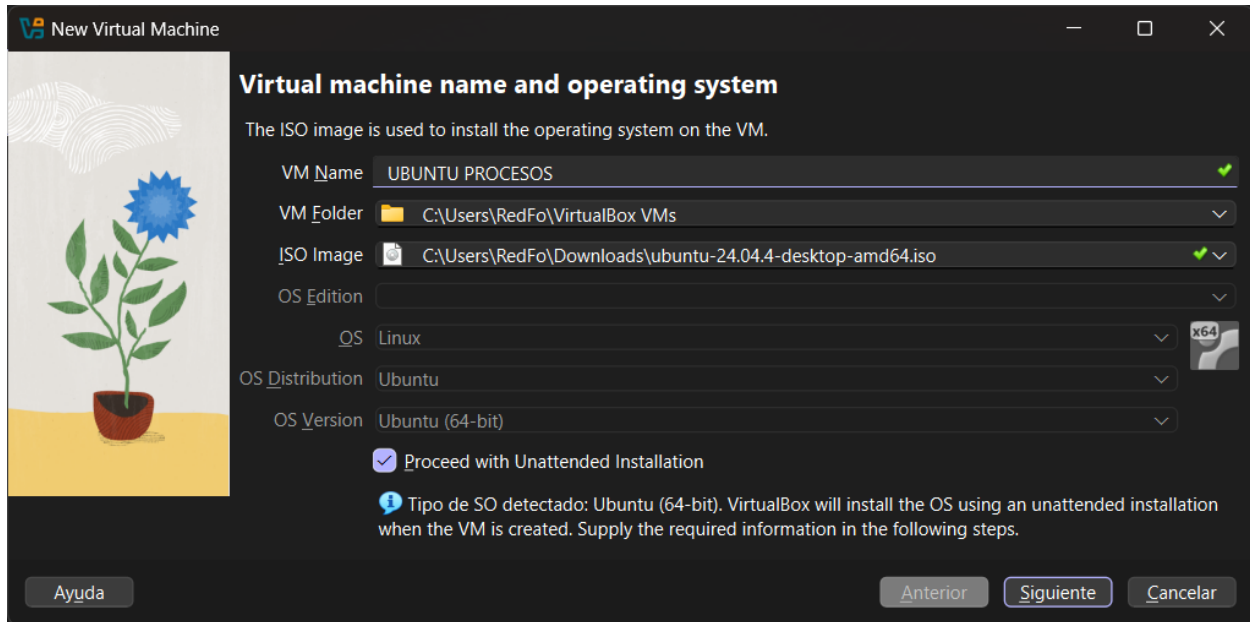
- VirtualBox — virtualizador donde se crean las máquinas virtuales.
- Ubuntu Server 22.04 LTS — sistema operativo del servidor.
- SDK de .NET 10 y MySQL 8 — para la ejecución directa de la Fase 1.
- Docker y Docker Compose — contenerización y orquestación.
- Vagrant — creación automática de la máquina virtual.
- Puppet — instalación automática de las herramientas en el servidor.

Fase 1. Instalación manual del servidor Linux

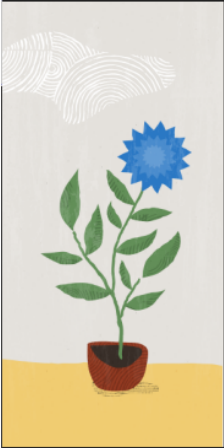
En esta fase se creó un servidor Linux y, sobre él, se instalaron manualmente MySQL y el SDK de .NET. La base de datos y la aplicación ReservaEspacios se ejecutaron directamente sobre el sistema operativo, sin contenedores.

1.1 Creación de la máquina virtual e instalación de Ubuntu Server

Paso 1. Se descargó la ISO de Ubuntu Server 22.04 LTS y se creó una máquina virtual en VirtualBox (2 GB de RAM, 2 CPU, 20 GB de disco), asignando la ISO a la unidad óptica.



New Virtual Machine



Set up unattended guest OS installation

Enter the information that will be required when the OS is installed.

User Name and Password

User Name

seiya

✓

Contraseña

••••

••••

Confirm Password

••••

••••

OS Installation Options

Product Key

✓

Host Name

UBUNTU-PROCESOS

✓

Domain Name

myguest.virtualbox.org

✓

☐ Instalar en segundo plano

☐ Install Guest Additions

Guest Additions ISO Image:

 C:\Program Files\Oracle\VirtualBox\VBBoxGuestAdditions.iso

Ayuda

Anterior

Siguiente

Cancelar

New Virtual Machine



Specify virtual hardware

Specify the VM's hardware. Resources allocated to the VM will not be available to the host when the VM is running.

Base Memory

4 MB

8192 MB

4096 MB

Number of CPUs

1 CPU

8 CPUs

4

Disk Size

4.00 MB

2.00 TB

25.00 GB

☐ Use EFI

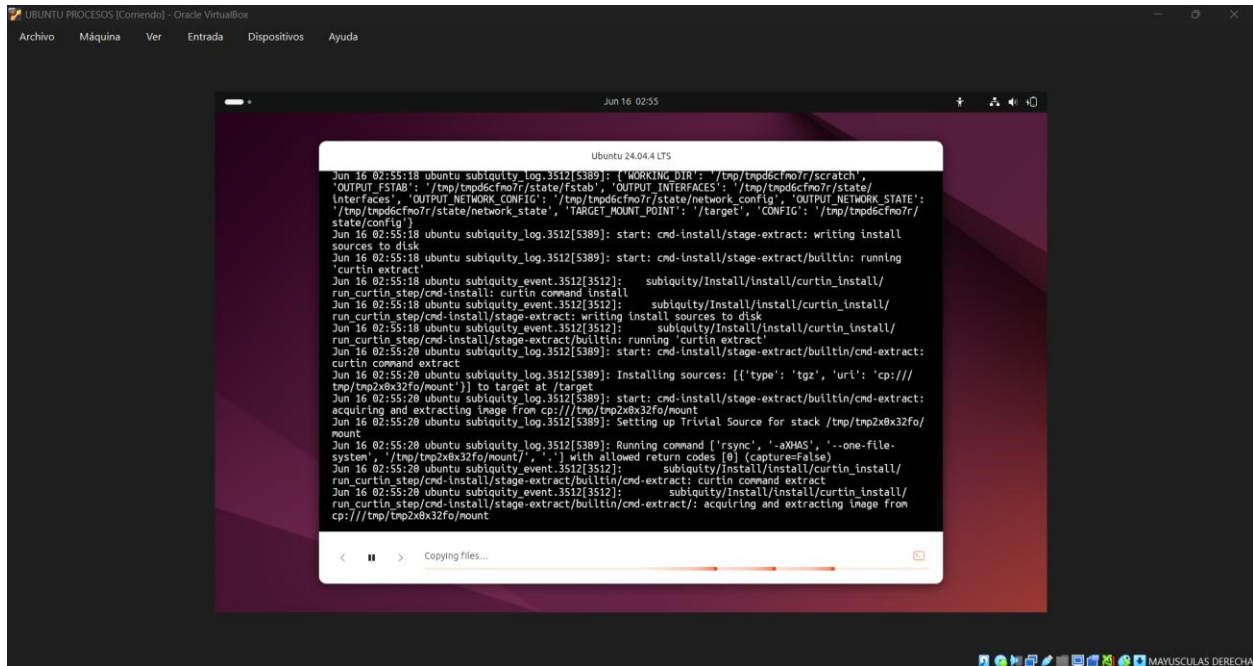
Ayuda

Anterior

Siguiente

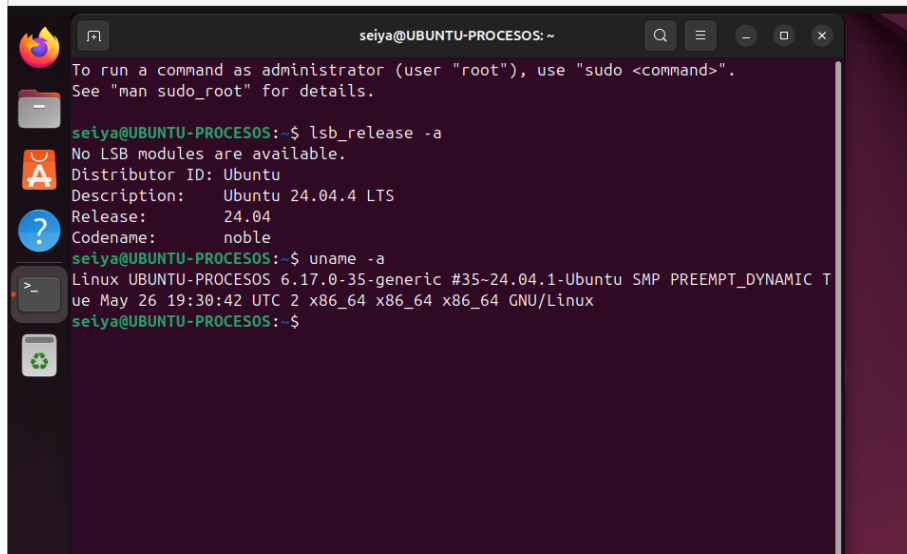
Cancelar

Paso 2. Se inició la máquina virtual y se instaló Ubuntu Server siguiendo el asistente, creando el usuario y habilitando OpenSSH.



Paso 3. Tras iniciar sesión, se verificó la versión del sistema operativo:

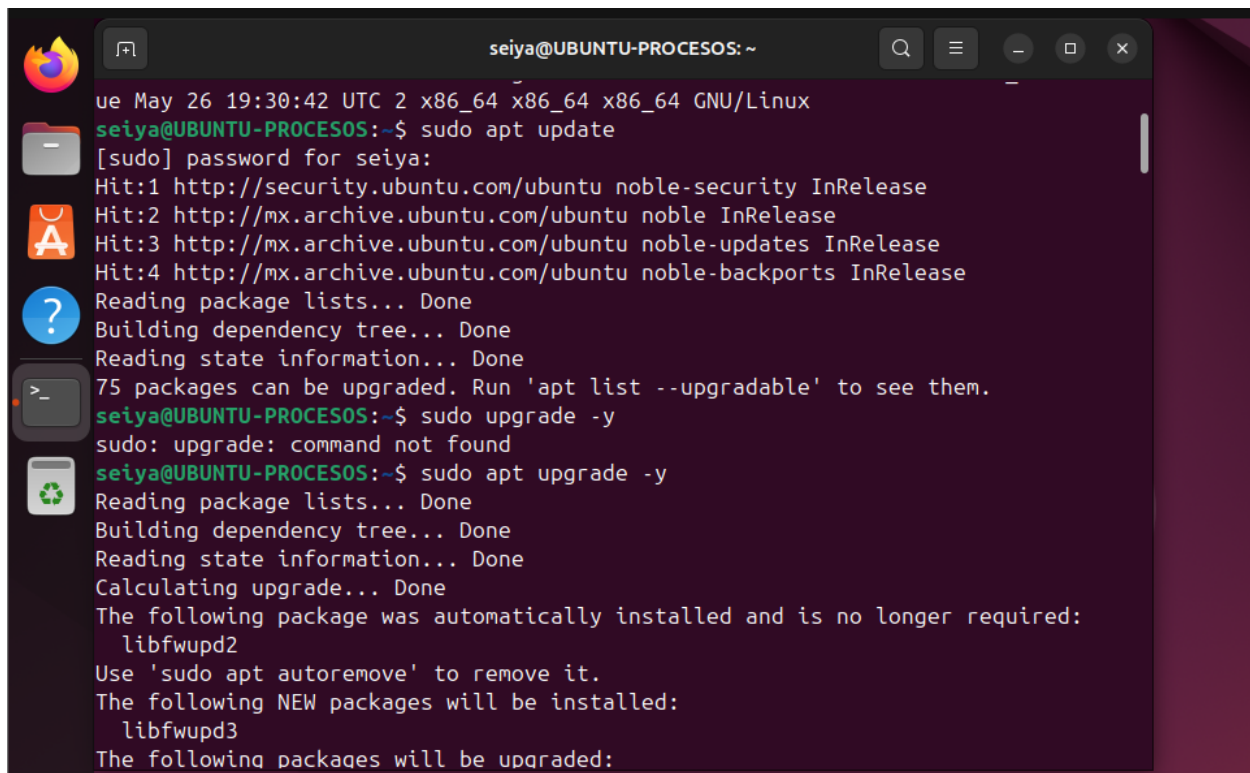
```
lsb_release -a
uname -a
```



1.2 Configuración inicial del servidor

Paso 4. Se actualizó la lista de paquetes y el sistema:

```
sudo apt update
sudo apt upgrade -y
```

A terminal window titled 'seiya@UBUNTU-PROCESOS: ~' with standard Ubuntu window controls. The terminal output shows the execution of 'sudo apt update' and 'sudo apt upgrade -y'. It lists sources like 'noble-security', 'noble', 'noble-updates', and 'noble-backports'. It reports that 75 packages can be upgraded. The upgrade process identifies 'libfwupd2' as no longer required and 'libfwupd3' as a new package to be installed. It also lists packages to be upgraded.

```
ue May 26 19:30:42 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
seiya@UBUNTU-PROCESOS:~$ sudo apt update
[sudo] password for seiya:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://mx.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://mx.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://mx.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
75 packages can be upgraded. Run 'apt list --upgradable' to see them.
seiya@UBUNTU-PROCESOS:~$ sudo upgrade -y
sudo: upgrade: command not found
seiya@UBUNTU-PROCESOS:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
The following NEW packages will be installed:
  libfwupd3
The following packages will be upgraded:
```

1.3 Instalación y configuración de la base de datos (MySQL)

Paso 5. Se instaló el servidor MySQL y se verificó que el servicio estuviera activo:

```
sudo apt install mysql-server -y
sudo systemctl status mysql
```

The screenshot shows a terminal window titled 'seiya@UBUNTU-PROCESOS: ~' with the following output:

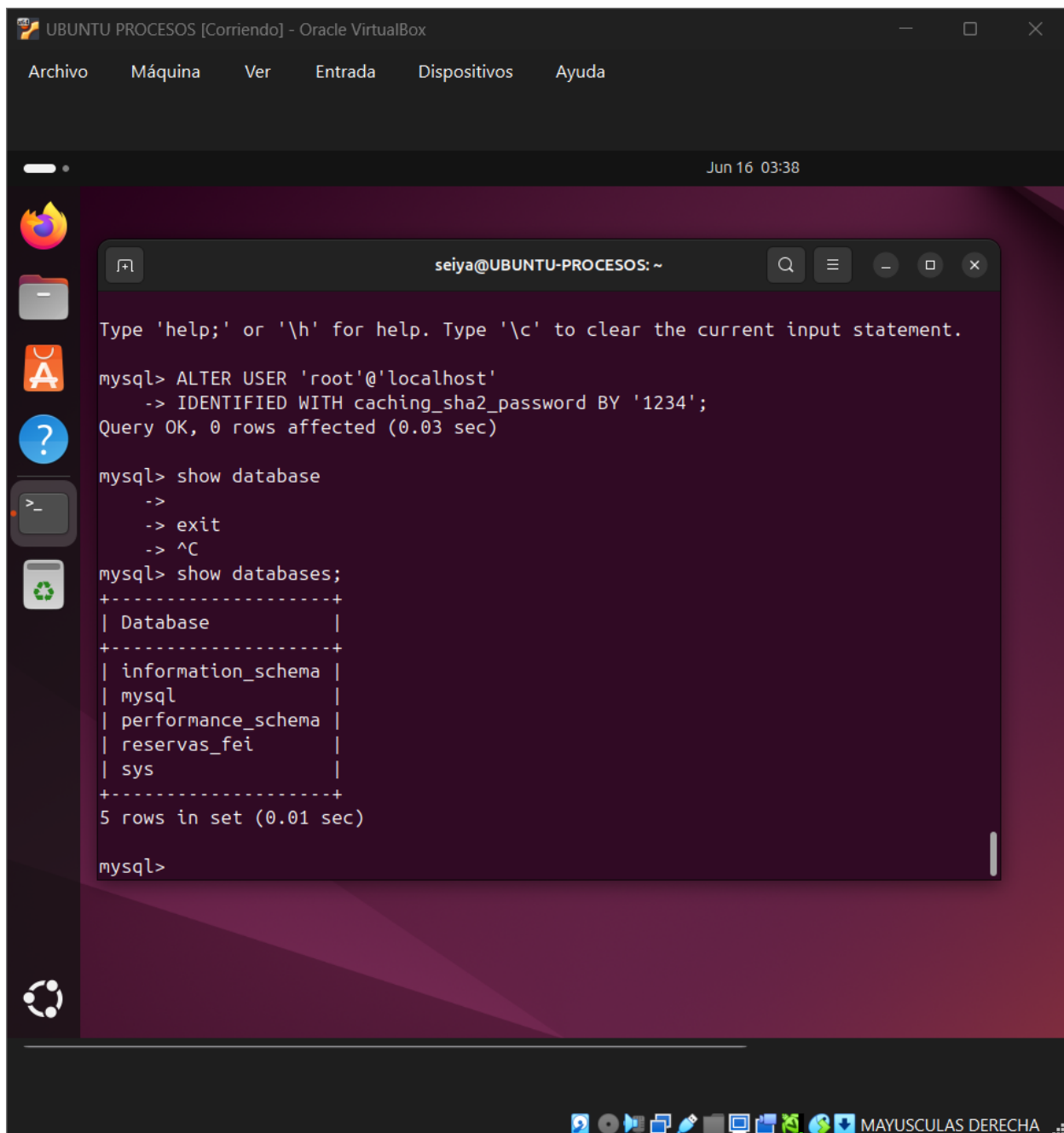
```
Renaming removed key_buffer and myisam-recover options (if present)
mysqld will log errors to /var/log/mysql/error.log
mysqld is running as pid 10876
Created symlink /etc/systemd/system/multi-user.target.wants/mysql.service → /usr/lib/systemd/system/mysql.service.
Setting up mysql-server (8.0.46-0ubuntu0.24.04.2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
seiya@UBUNTU-PROCESOS:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: en>
   Active: active (running) since Tue 2026-06-16 03:25:46 UTC; 17s ago
     Process: 11158 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=>
    Main PID: 11166 (mysqld)
      Status: "Server is operational"
        Tasks: 38 (limit: 4597)
      Memory: 364.0M (peak: 378.3M)
         CPU: 1.729s
       CGroup: /system.slice/mysql.service
               └─11166 /usr/sbin/mysqld

Jun 16 03:25:45 UBUNTU-PROCESOS systemd[1]: Starting mysql.service - MySQL Commu>
Jun 16 03:25:46 UBUNTU-PROCESOS systemd[1]: Started mysql.service - MySQL Commu>
lines 1-14/14 (END)
```

Paso 6. Se creó la base de datos «reservas_fei» y se asignó a root la contraseña usada por el proyecto:

```
sudo mysql

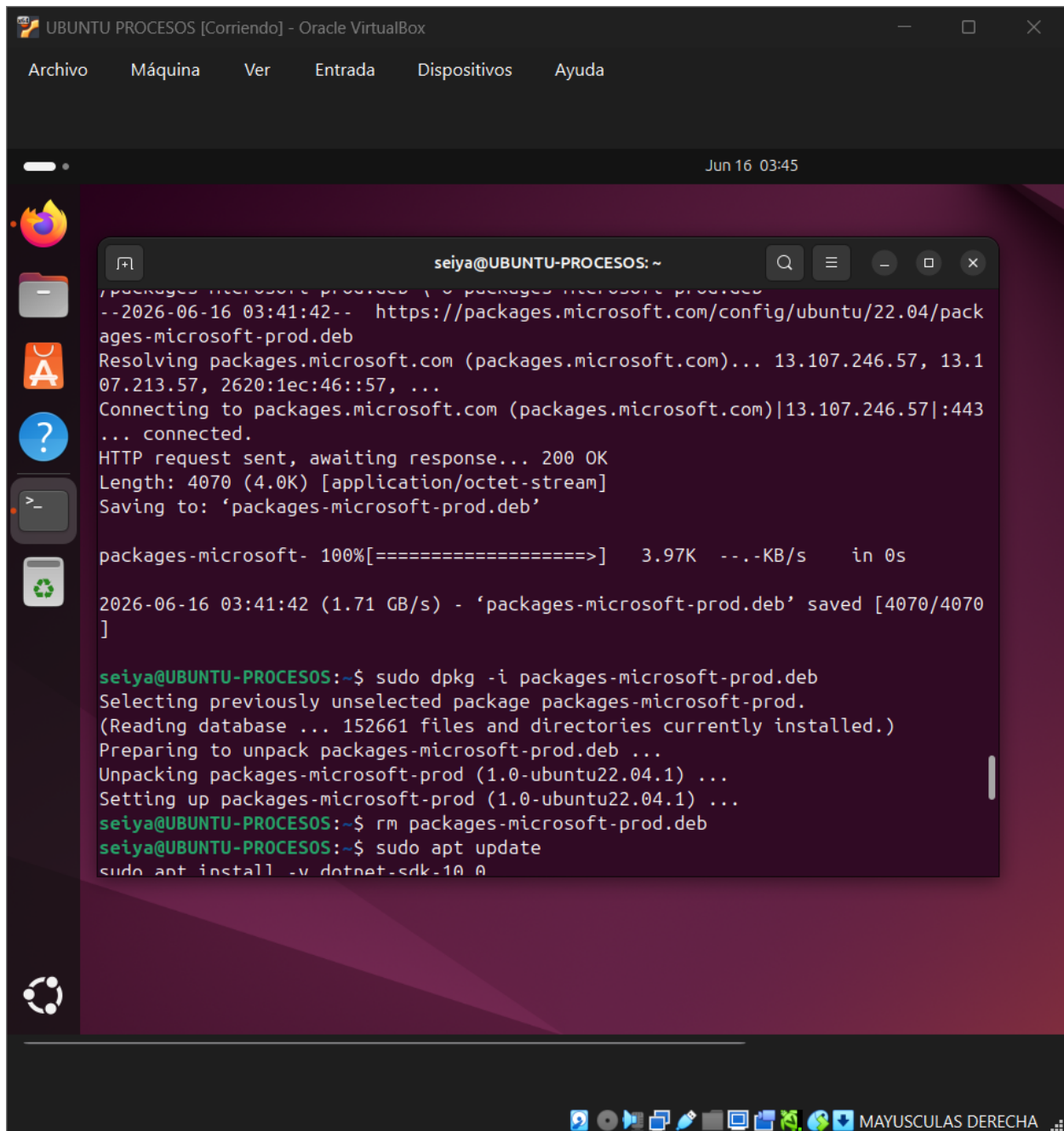
ALTER USER 'root'@'localhost' IDENTIFIED WITH caching_sha2_password BY '1234';
CREATE DATABASE reservas_fei;
FLUSH PRIVILEGES;
EXIT;
```



1.4 Instalación del SDK de .NET 10

Paso 7. Se agregó el repositorio de Microsoft y se instaló el SDK de .NET 10:

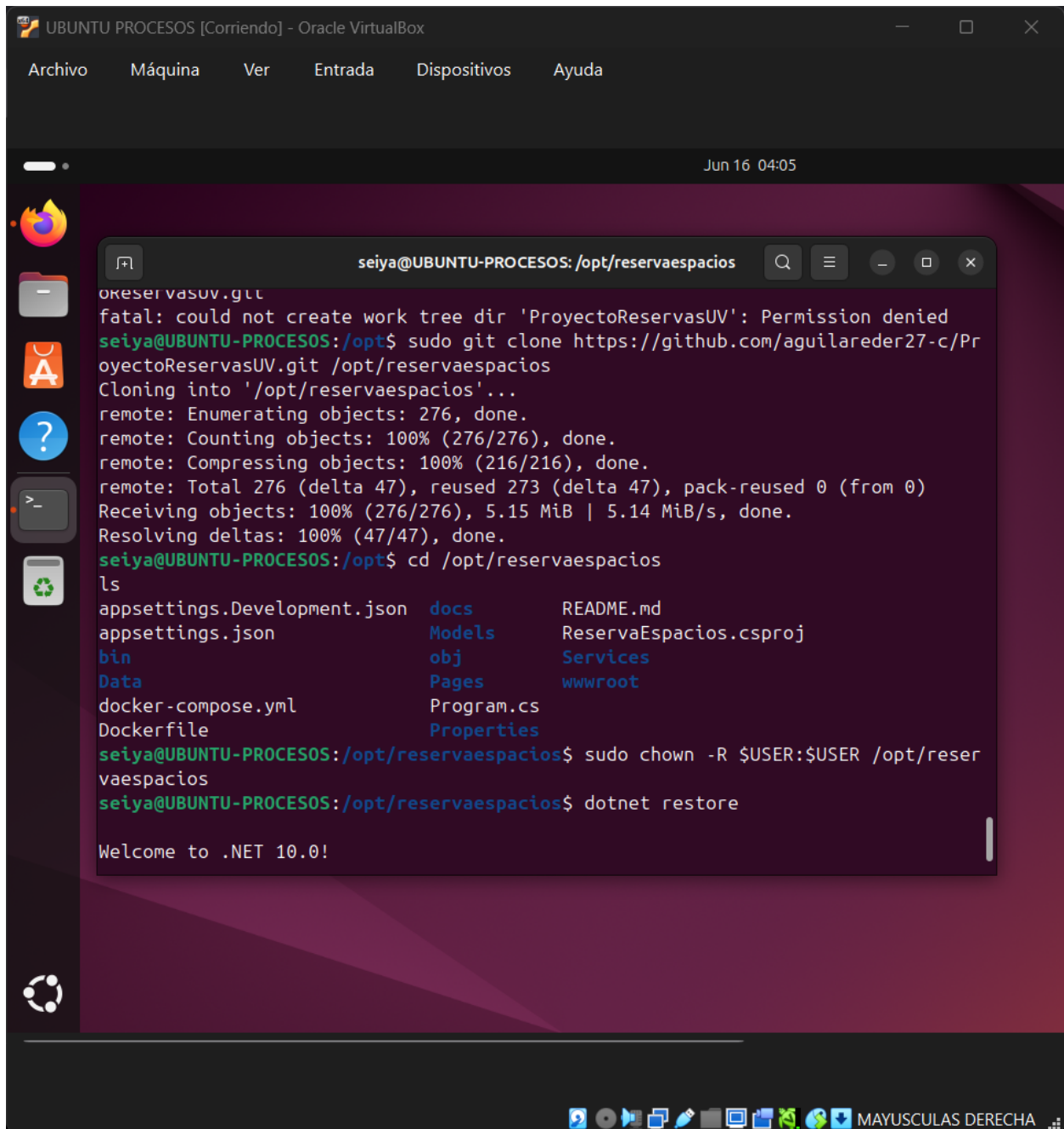
```
wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb \
-o packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
rm packages-microsoft-prod.deb
sudo apt update
sudo apt install -y dotnet-sdk-10.0
dotnet --version
```



1.5 Despliegue del sistema y creación del esquema

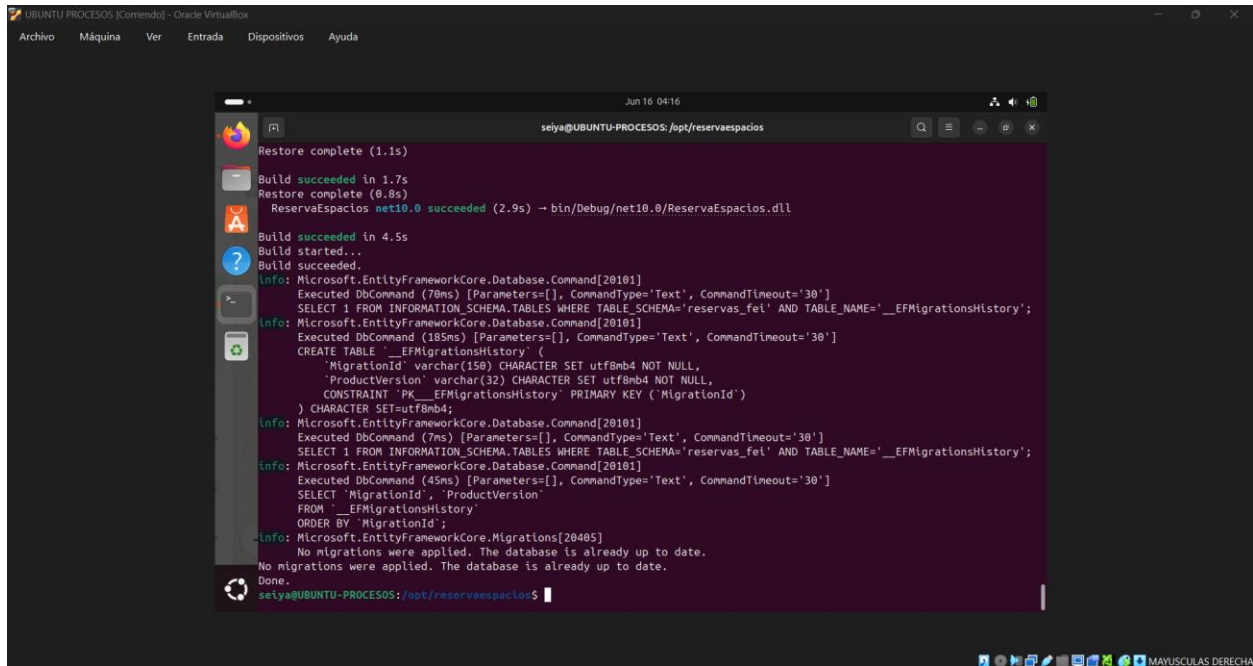
Paso 8. Se copió el código del proyecto al servidor, en /opt/reservaespacios, y se restauraron las dependencias:

```
git clone <URL > /opt/reservaespacios  
cd /opt/reservaespacios  
dotnet restore
```



Paso 9. Se instalaron las herramientas de EF Core, se creó la migración inicial y se aplicó para generar las tablas en la base de datos:

```
dotnet tool install --global dotnet-ef
export PATH="$PATH:$HOME/.dotnet/tools"
dotnet ef migrations add CreacionInicial
dotnet ef database update
```

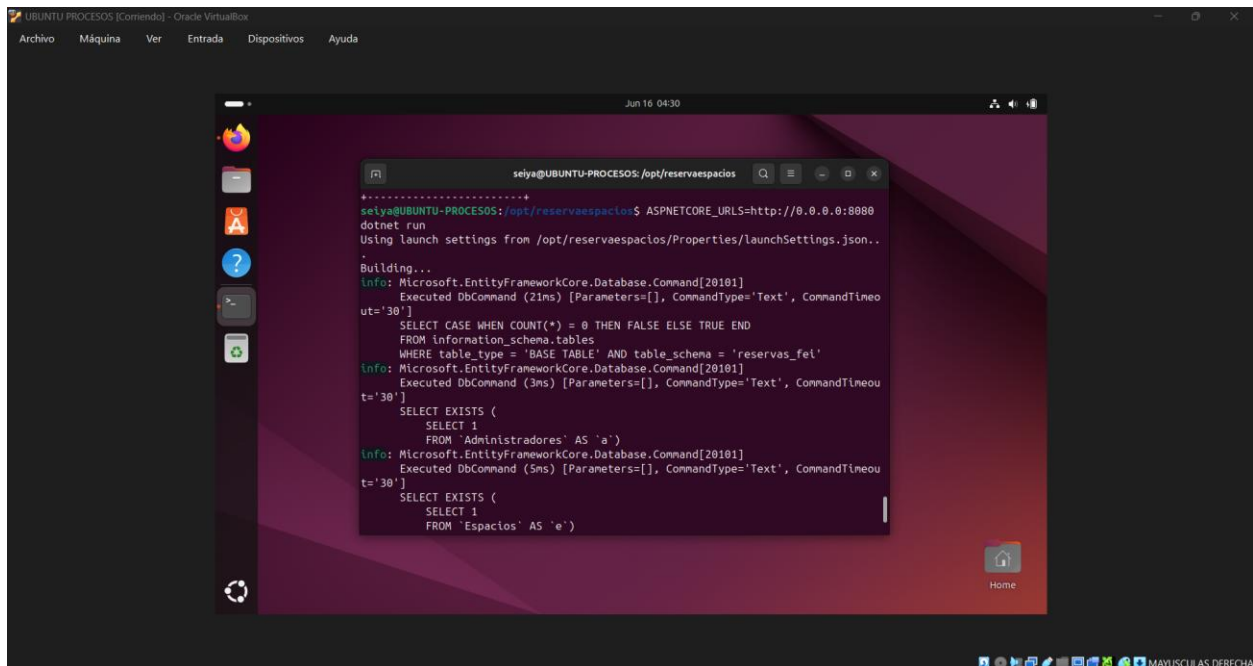


```
Restore complete (1.1s)
Build succeeded in 1.7s
Restore complete (0.8s)
ReservaEspacios net10.0 succeeded (2.9s) -> bin/Debug/net10.0/ReservaEspacios.dll
Build succeeded in 4.5s
Build started...
Build succeeded.
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (7ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA='reservas_fel' AND TABLE_NAME='__EFMigrationsHistory';
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (185ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE '__EFMigrationsHistory' (
        'MigrationId' varchar(150) CHARACTER SET utf8mb4 NOT NULL,
        'ProductVersion' varchar(32) CHARACTER SET utf8mb4 NOT NULL,
        CONSTRAINT 'PK__EFMigrationsHistory' PRIMARY KEY ('MigrationId')
      ) CHARACTER SET=utf8mb4;
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (7ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA='reservas_fel' AND TABLE_NAME='__EFMigrationsHistory';
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (45ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 'MigrationId', 'ProductVersion'
      FROM '__EFMigrationsHistory'
      ORDER BY 'MigrationId';
Info: Microsoft.EntityFrameworkCore.Migrations[20405]
      No migrations were applied. The database is already up to date.
Done.
seiya@UBUNTU-PROCESOS:/opt/reservaespacios$
```

1.6 Ejecución y verificación

Paso 10. Se ejecutó el sistema indicando que escuchara en todas las interfaces, en el puerto 8080:

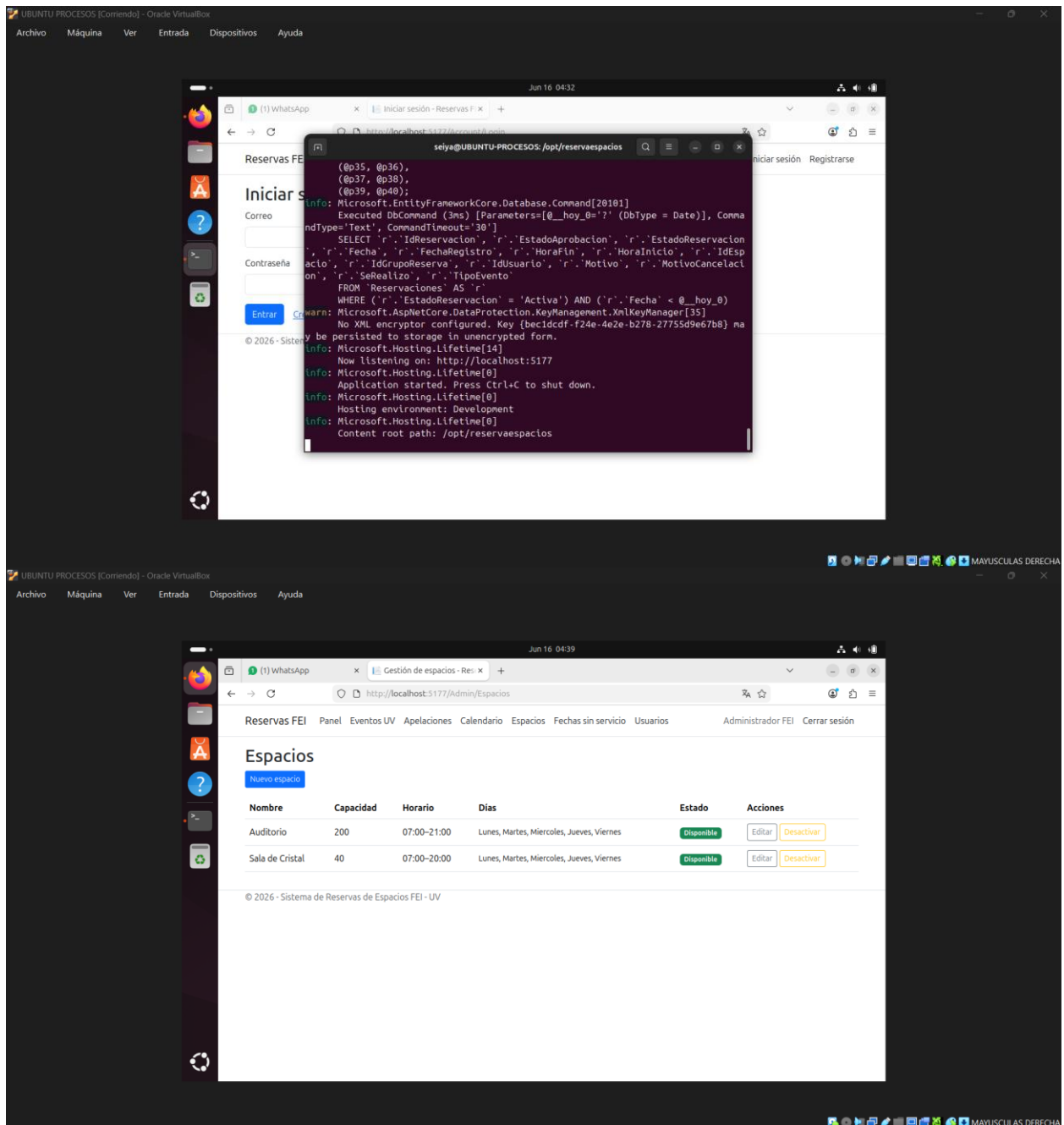
```
ASPNETCORE_URLS=http://0.0.0.0:8080 dotnet run
```



```
seiya@UBUNTU-PROCESOS:/opt/reservaespacios$ ASPNETCORE_URLS=http://0.0.0.0:8080 dotnet run
Using launch settings from /opt/reservaespacios/Properties/launchSettings.json..
Building...
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (21ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT CASE WHEN COUNT(*) = 0 THEN FALSE ELSE TRUE END
      FROM information_schema.tables
      WHERE table_type = 'BASE TABLE' AND table_schema = 'reservas_fel'
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
        SELECT 1
        FROM 'Administradores' AS 'a'
      )
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (5ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
        SELECT 1
        FROM 'Espacios' AS 'e'
      )
```

Paso 11. Se abrió el sistema en el navegador usando la IP del servidor; se inició sesión con la cuenta de administrador sembrada (admin@fei.uv.mx / Admin123!):

http://<>:8080



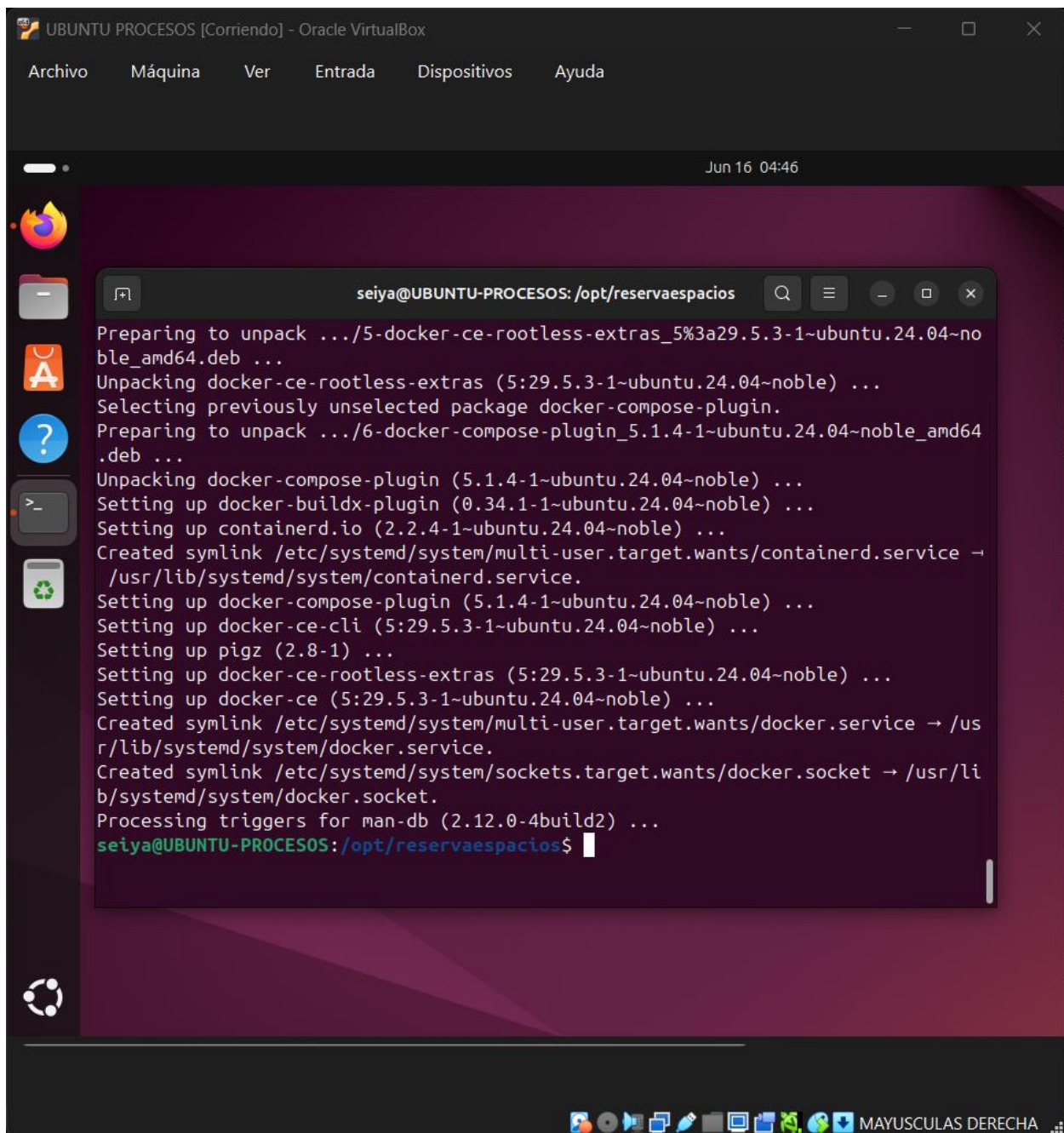
Fase 2. Despliegue con Docker y Docker Compose

En esta fase se empaquetó el sistema ReservaEspacios en un contenedor Docker y la base de datos MySQL en otro, y ambos se desplegaron sobre el servidor Linux mediante Docker Compose. La aplicación y la base de datos se levantaron con un solo comando.

2.1 Instalación de Docker y Docker Compose

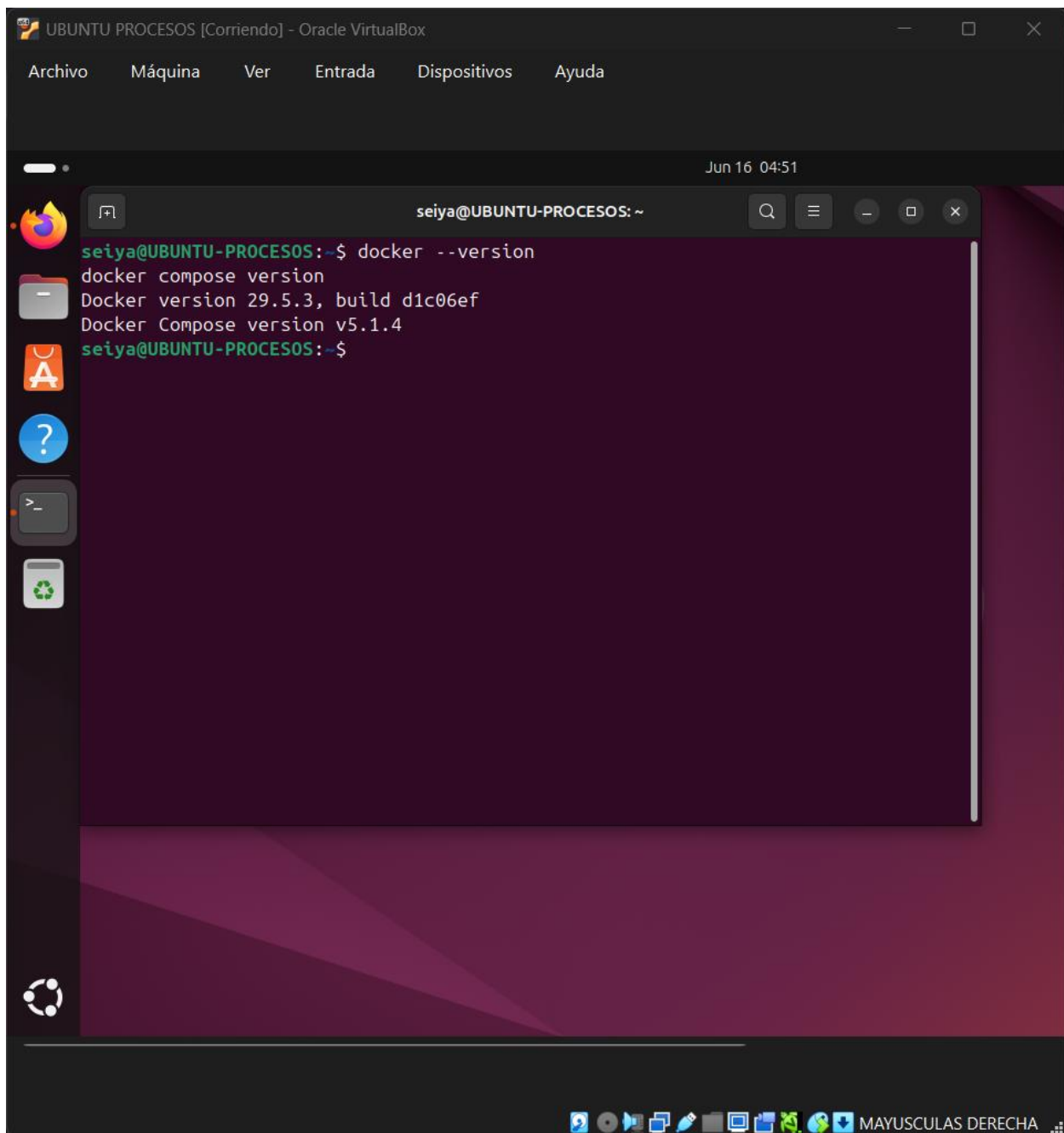
Paso 1. Se instaló Docker Engine y el plugin de Docker Compose sobre el servidor:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```



Paso 2. Se verificó la instalación y se agregó el usuario al grupo docker:

```
docker --version
docker compose version
sudo docker run hello-world
sudo usermod -aG docker $USER
```



2.2 Estructura del proyecto

En el servidor, el proyecto quedó en `/opt/reservaespacios`, con el `Dockerfile`, el `docker-compose.yml` y la carpeta `db/` junto al código del sistema:

```
/opt/reservaespacios/      (carpeta del proyecto en el servidor)
├── Dockerfile              # imagen del sistema
├── docker-compose.yml      # orquestación db + web
├── db/
│   └── init.sql            # esquema generado desde EF Core
├── ReservaEspacios.csproj
└── Program.cs
```

```
└─ Data/   Models/   Services/
└─ ... (resto del código)
```

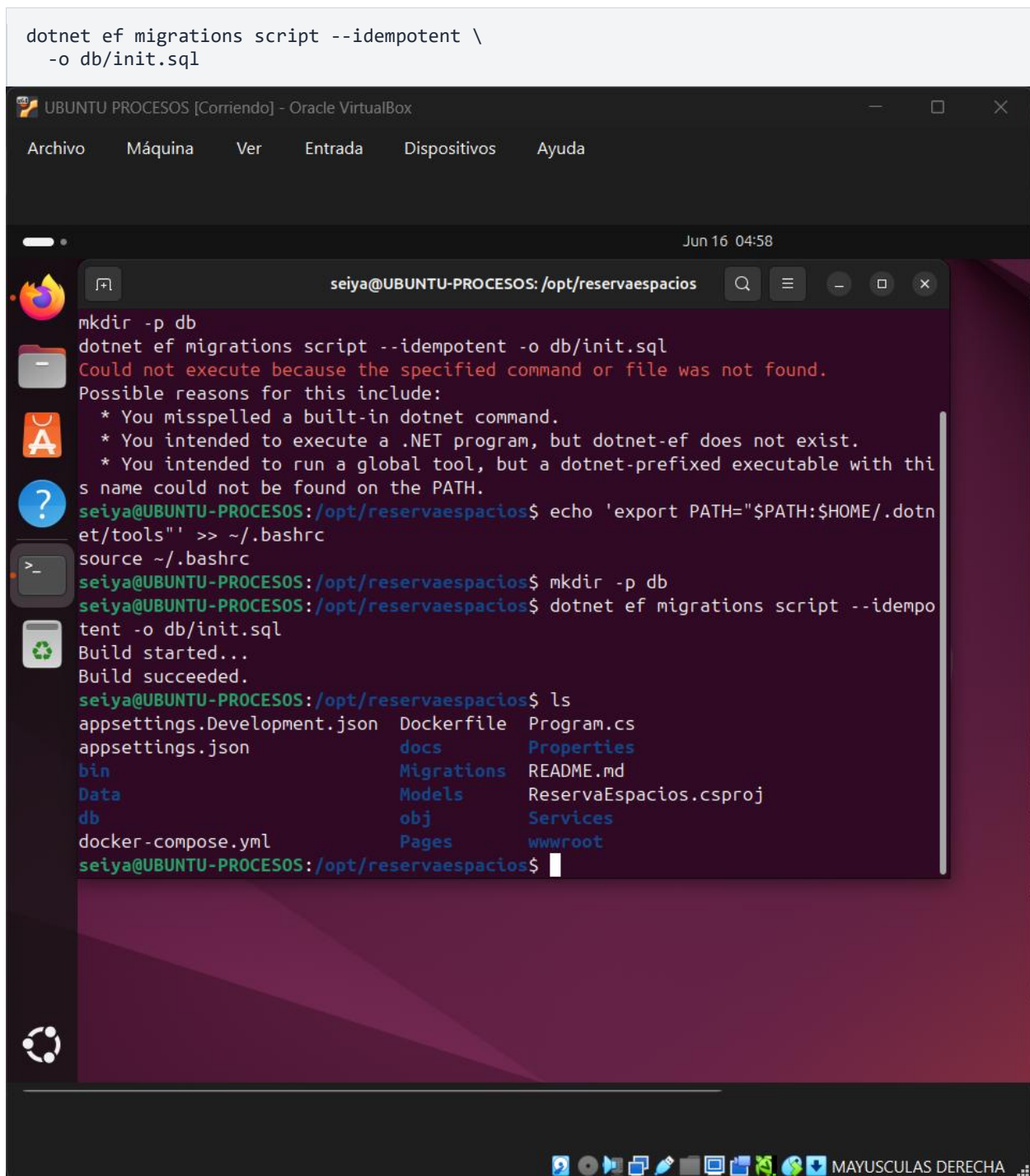
The screenshot shows a terminal window titled "UBUNTU PROCESOS [Corriendo] - Oracle VirtualBox". The terminal output is as follows:

```
seiya@UBUNTU-PROCESOS: /opt/reservaespacios
mkdir -p db
dotnet ef migrations script --idempotent -o db/init.sql
Could not execute because the specified command or file was not found.
Possible reasons for this include:
 * You misspelled a built-in dotnet command.
 * You intended to execute a .NET program, but dotnet-ef does not exist.
 * You intended to run a global tool, but a dotnet-prefixed executable with this name could not be found on the PATH.
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ echo 'export PATH="$PATH:$HOME/.dotnet/tools"' >> ~/.bashrc
source ~/.bashrc
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ mkdir -p db
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ dotnet ef migrations script --idempotent -o db/init.sql
Build started...
Build succeeded.
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ ls
appsettings.Development.json  Dockerfile  Program.cs
appsettings.json              docs        Properties
bin                            Migrations  README.md
Data                           Models      ReservaEspacios.csproj
db                             obj         Services
docker-compose.yml            Pages       wwwroot
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$
```

2.3 Generación del esquema de la base de datos

Se generó un script SQL a partir de las migraciones de EF Core. Este script crea las tablas dentro del contenedor de MySQL en su primer arranque; es necesario porque el sistema usa EnsureCreated, que no crea las tablas si la base de datos ya existe:

```
dotnet ef migrations script --idempotent \
-o db/init.sql
```



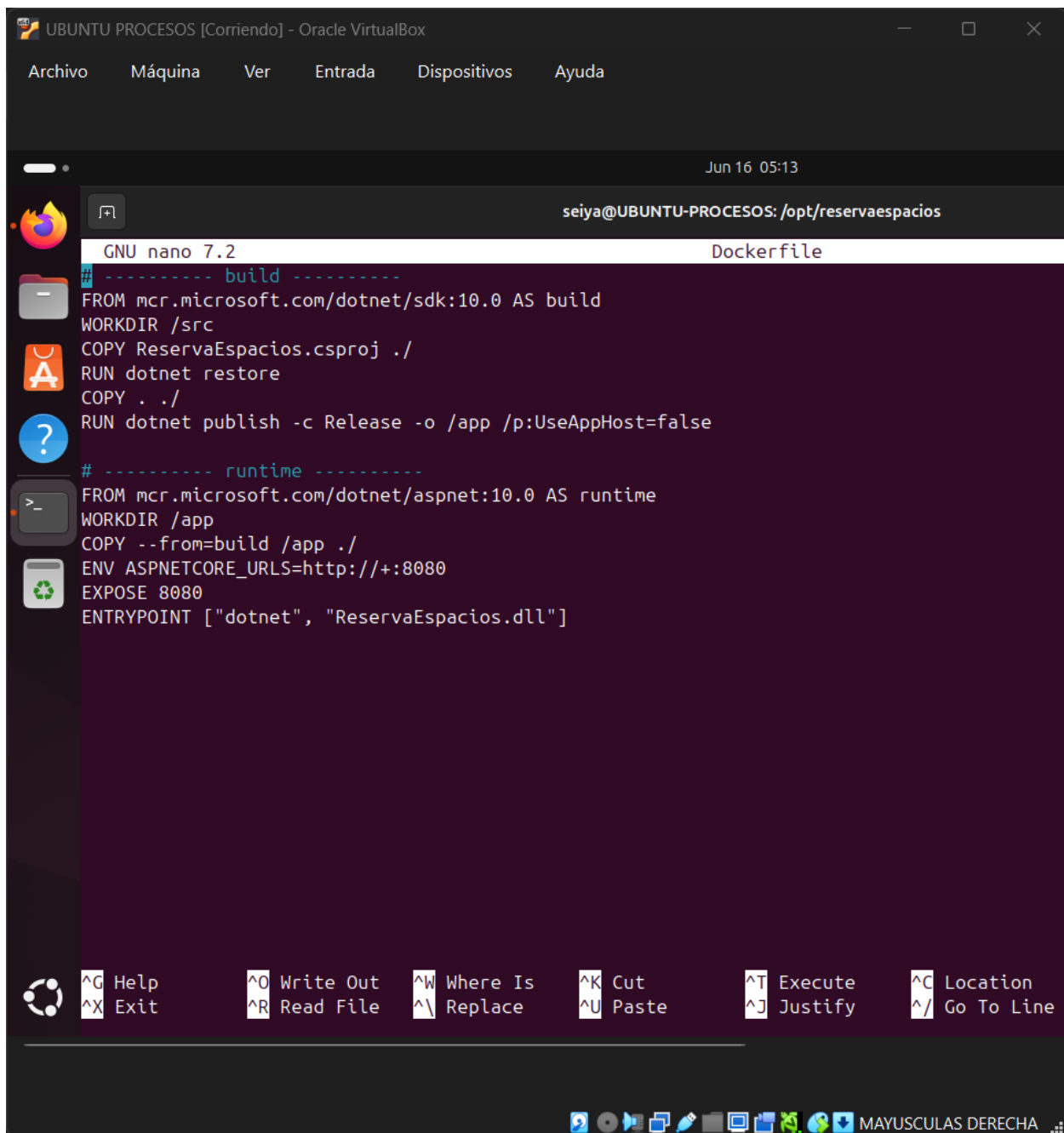
```
seiya@UBUNTU-PROCESOS: /opt/reservaespacios
mkdir -p db
dotnet ef migrations script --idempotent -o db/init.sql
Could not execute because the specified command or file was not found.
Possible reasons for this include:
 * You misspelled a built-in dotnet command.
 * You intended to execute a .NET program, but dotnet-ef does not exist.
 * You intended to run a global tool, but a dotnet-prefixed executable with this name could not be found on the PATH.
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ echo 'export PATH="$PATH:$HOME/.dotnet/tools"' >> ~/.bashrc
source ~/.bashrc
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ mkdir -p db
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ dotnet ef migrations script --idempotent -o db/init.sql
Build started...
Build succeeded.
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$ ls
appsettings.Development.json  Dockerfile  Program.cs
appsettings.json              docs        Properties
bin                            Migrations  README.md
Data                           Models      ReservaEspacios.csproj
db                             obj         Services
docker-compose.yml            Pages       wwwroot
seiya@UBUNTU-PROCESOS: /opt/reservaespacios$
```

2.4 Dockerfile del sistema

El Dockerfile del proyecto usa dos etapas: una compila el proyecto con el SDK de .NET y otra lo ejecuta con la imagen ligera de ASP.NET Core.

```
# Etapa 1: compilación
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
```

```
COPY . .  
RUN dotnet restore  
RUN dotnet publish -c Release -o /app/publish /p:UseAppHost=false  
  
# Etapa 2: ejecución  
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime  
WORKDIR /app  
COPY --from=build /app/publish .  
ENV ASPNETCORE_HTTP_PORTS=8080  
EXPOSE 8080  
ENTRYPOINT ["dotnet", "ReservaEspacios.dll"]
```



```
UBUNTU PROCESOS [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Jun 16 05:13
seiya@UBUNTU-PROCESOS: /opt/reservaespacios

GNU nano 7.2 Dockerfile
# ----- build -----
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY ReservaEspacios.csproj ./
RUN dotnet restore
COPY . ./
RUN dotnet publish -c Release -o /app /p:UseAppHost=false

# ----- runtime -----
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app ./
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "ReservaEspacios.dll"]

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute   ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify   ^_ Go To Line

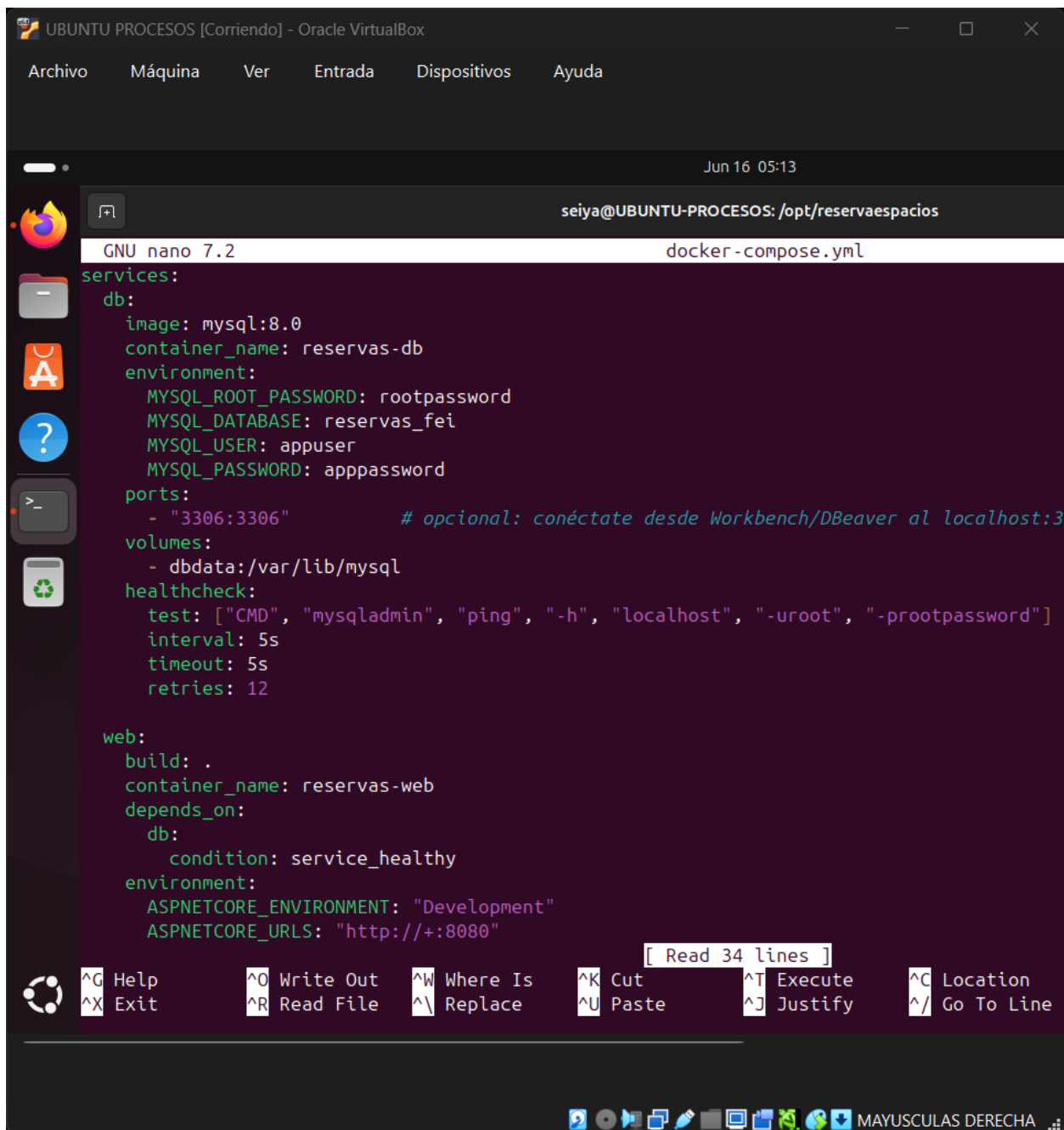
MAYUSCULAS DERECHA
```

2.5 Archivo docker-compose.yml

El docker-compose.yml define dos servicios: «db» (MySQL) y «web» (el sistema). El servicio web espera con un healthcheck a que la base esté lista y se conecta usando el nombre del servicio «db» como servidor, con el usuario «appuser».

```
services:
  db:
    image: mysql:8.0
    container_name: reservas-db
    environment:
```

```
    MYSQL_ROOT_PASSWORD: rootpassword
    MYSQL_DATABASE: reservas_fei
    MYSQL_USER: appuser
    MYSQL_PASSWORD: apppassword
  ports:
    - "3306:3306"
  volumes:
    - dbdata:/var/lib/mysql
    - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-prootpassword"]
    interval: 5s
    timeout: 5s
    retries: 12
  web:
    build: .
    container_name: reservas-web
    depends_on:
      db:
        condition: service_healthy
    environment:
      ASPNETCORE_ENVIRONMENT: "Development"
      ASPNETCORE_URLS: "http://+:8080"
      ConnectionStrings__Default:
        "Server=db;Port=3306;Database=reservas_fei;User=appuser;Password=apppassword;AllowPublicKe
yRetrieval=True;SslMode=None;"
    ports:
      - "8080:8080"
  volumes:
    dbdata:
```



```
UBUNTU PROCESOS [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Jun 16 05:13
seiya@UBUNTU-PROCESOS: /opt/reservaespacios
GNU nano 7.2  docker-compose.yml

services:
  db:
    image: mysql:8.0
    container_name: reservas-db
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: reservas_fei
      MYSQL_USER: appuser
      MYSQL_PASSWORD: apppassword
    ports:
      - "3306:3306"          # opcional: conéctate desde Workbench/DBever al localhost:3
    volumes:
      - dbdata:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-prootpassword"]
      interval: 5s
      timeout: 5s
      retries: 12

  web:
    build: .
    container_name: reservas-web
    depends_on:
      db:
        condition: service_healthy
    environment:
      ASPNETCORE_ENVIRONMENT: "Development"
      ASPNETCORE_URLS: "http://+:8080"

[ Read 34 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
^_           ^C Location   ^/ Go To Line

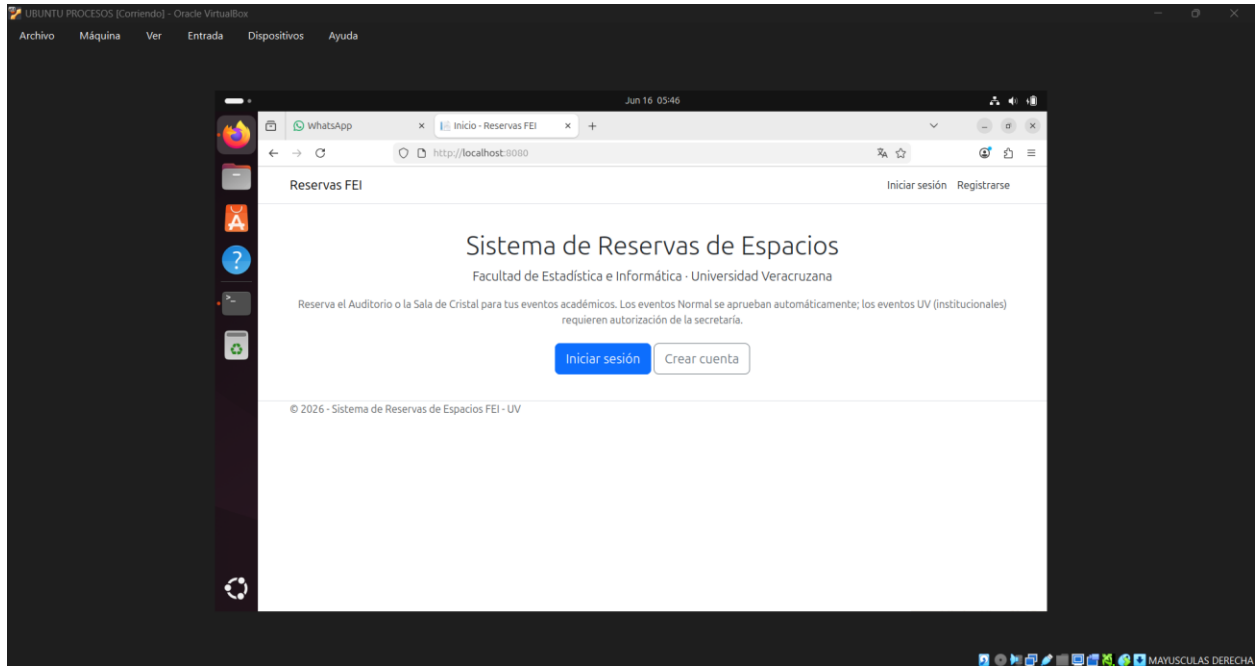
MAYUSCULAS DERECHA
```

2.6 Construcción, levantamiento y verificación

Paso 3. Se detuvo el MySQL del host para liberar el puerto 3306 y se construyeron y levantaron los contenedores:

```
sudo systemctl stop mysql
sudo docker compose down -v
sudo docker compose up -d --build
```


`http://<IP-del-servidor>:8080`



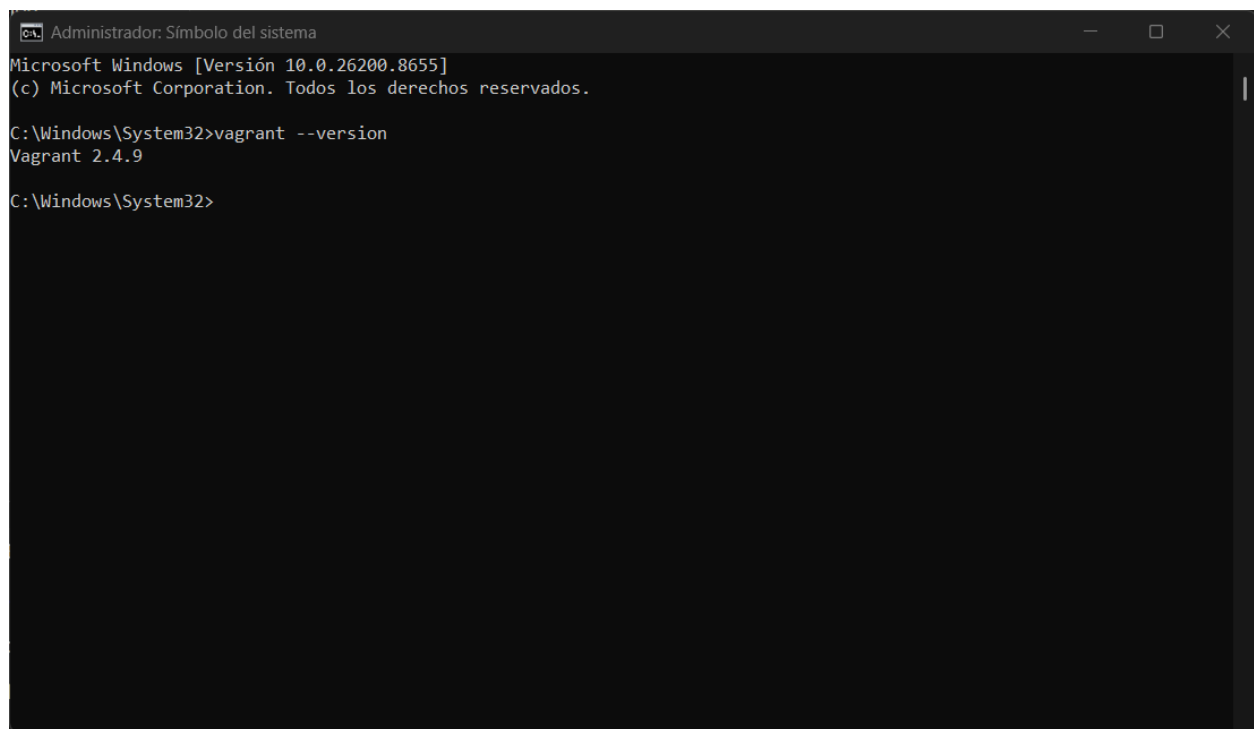
Fase 3. Servidor automático con Vagrant y Puppet

En esta fase se reprodujo el mismo despliegue en contenedores de la Fase 2, pero sobre un nuevo servidor Linux creado automáticamente con Vagrant. Las herramientas (Docker y Docker Compose) se instalaron mediante Puppet y, al final, los contenedores se levantaron solos. Todos los comandos de esta fase se ejecutaron en el equipo anfitrión (Windows).

3.1 Instalación de Vagrant en el equipo anfitrión

Paso 1. En el equipo anfitrión (Windows) se instaló Vagrant y se verificó su versión (VirtualBox ya estaba instalado):

```
vagrant --version
```



```
Administrador: Símbolo del sistema
Microsoft Windows [Versión 10.0.26200.8655]
(c) Microsoft Corporation. Todos los derechos reservados.

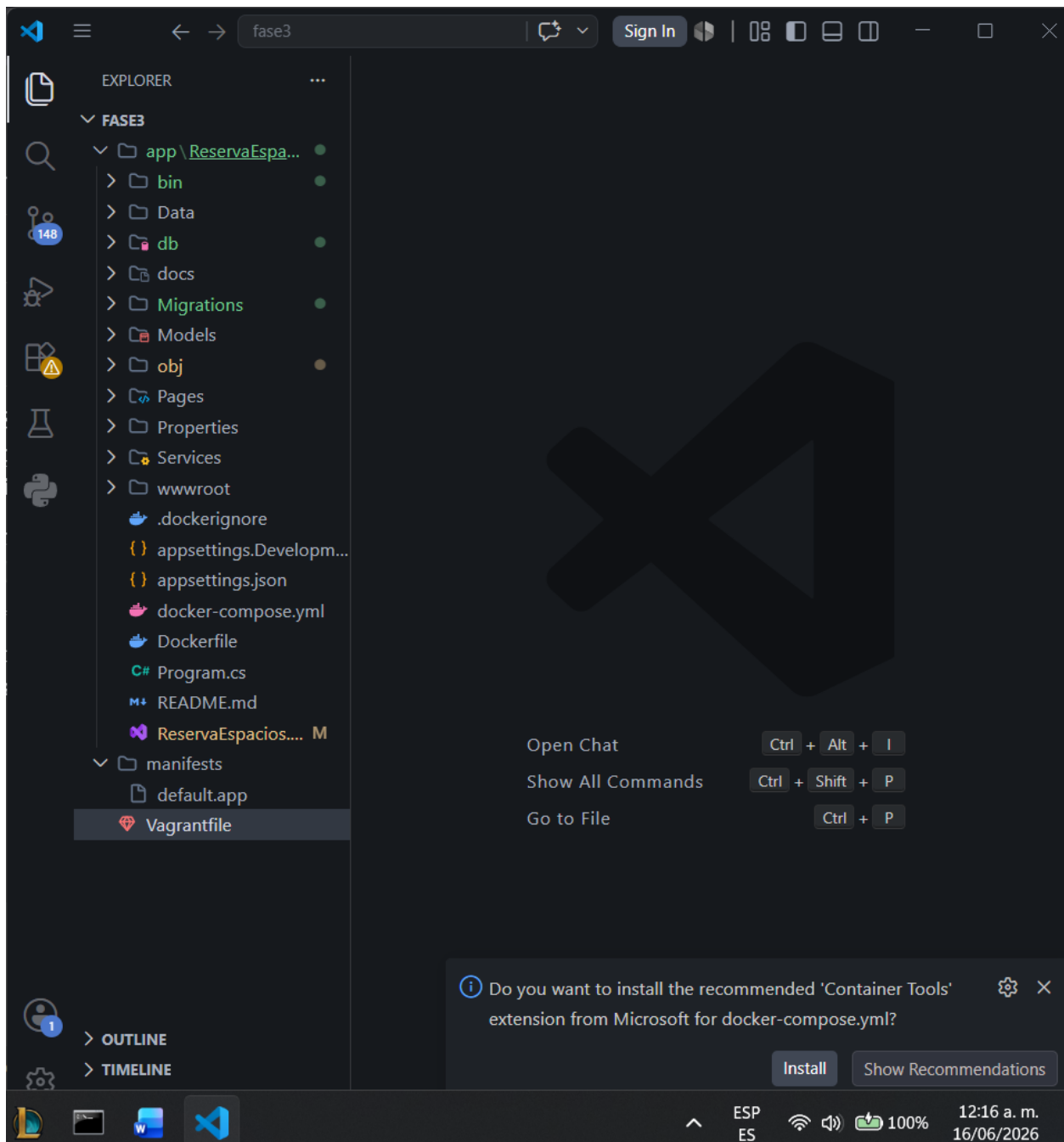
C:\Windows\System32>vagrant --version
Vagrant 2.4.9

C:\Windows\System32>
```

3.2 Estructura del proyecto

En el equipo anfitrión se creó la carpeta fase3 con la siguiente estructura. Dentro de «app» se colocó una copia del proyecto con su Dockerfile, docker-compose.yml y db/init.sql:

```
fase3/
├── Vagrantfile           # define y crea el servidor
├── manifests/
│   └── default.pp       # Puppet: instala Docker
└── app/
    ├── ReservaEspacios/ # copia del proyecto
    │   ├── Dockerfile
    │   ├── docker-compose.yml
    │   ├── db/init.sql
    │   └── ... (código del sistema)
```



3.3 Vagrantfile utilizado

El Vagrantfile define el servidor Linux que se creará automáticamente, su red, la carpeta compartida con la aplicación, e indica que se aprovisiona con Puppet y que al final se levanten los contenedores.

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/jammy64"
  config.vm.hostname = "servidor-reservas"
  config.vm.boot_timeout = 600
```

```

config.vm.network "private_network", ip: "192.168.56.10"
config.vm.network "forwarded_port", guest: 8080, host: 8080

config.vm.provider "virtualbox" do |vb|
  vb.gui = true
  vb.memory = 4096
  vb.cpus = 2
end

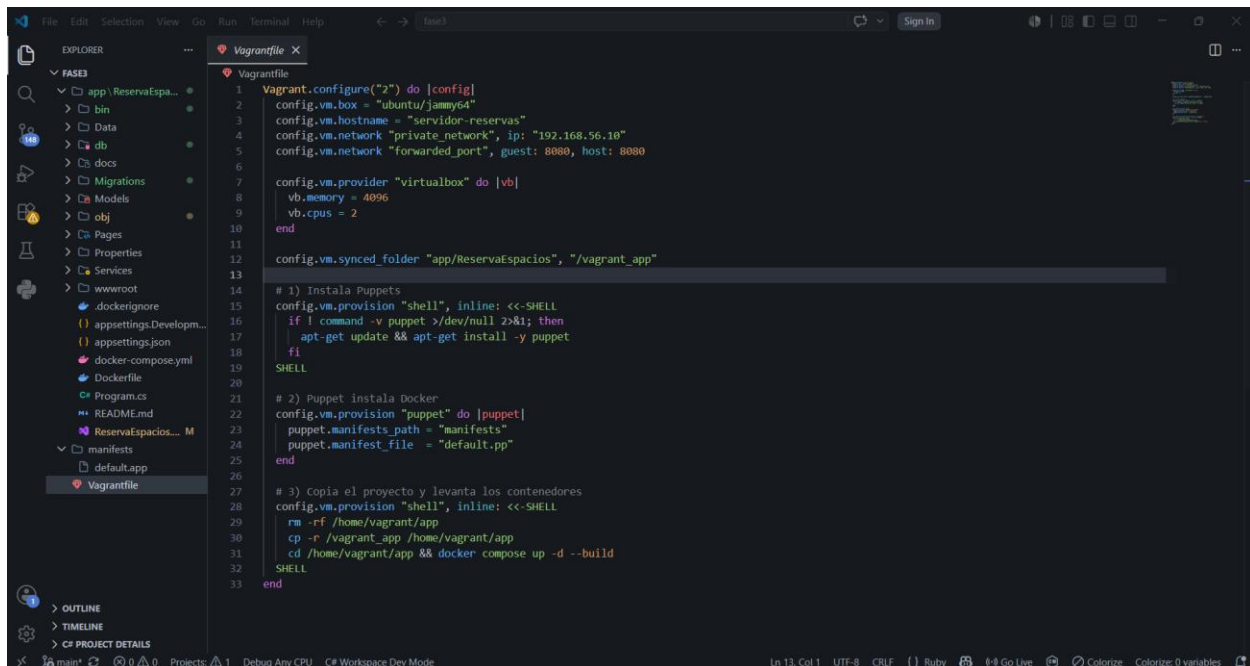
config.vm.synced_folder "app/ReservaEspacios", "/vagrant_app"

# 1) Instala Puppet
config.vm.provision "shell", inline: <<-SHELL
  if ! command -v puppet >/dev/null 2>&1; then
    apt-get update && apt-get install -y puppet
  fi
SHELL

# 2) Puppet instala Docker y Docker Compose
config.vm.provision "puppet" do |puppet|
  puppet.manifests_path = "manifests"
  puppet.manifest_file = "default.pp"
end

# 3) Copia el proyecto y levanta los contenedores
config.vm.provision "shell", inline: <<-SHELL
  rm -rf /home/vagrant/app
  cp -r /vagrant_app /home/vagrant/app
  cd /home/vagrant/app && docker compose up -d --build
SHELL
end
SHELL
end

```



3.4 Manifiesto de Puppet

El archivo manifests/default.pp indica a Puppet qué instalar: Docker y el plugin de Docker Compose, dejando el servicio en marcha.

```
# Actualizar el índice de paquetes
exec { 'apt-update':
  command => '/usr/bin/apt-get update',
}

# Paquetes previos
package { ['ca-certificates', 'curl', 'gnupg']:
  ensure => installed,
  require => Exec['apt-update'],
}

# Llave GPG oficial de Docker
exec { 'docker-gpg':
  command => '/bin/bash -c "install -m 0755 -d /etc/apt/keyrings && \
    curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    gpg --dearmor -o /etc/apt/keyrings/docker.gpg && \
    chmod a+r /etc/apt/keyrings/docker.gpg"',
  creates => '/etc/apt/keyrings/docker.gpg',
  require => Package['curl'],
}

# Repositorio de Docker
exec { 'docker-repo':
  command => '/bin/bash -c "echo deb [arch=amd64 \
    signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu jammy stable > \
    /etc/apt/sources.list.d/docker.list && apt-get update"',
  creates => '/etc/apt/sources.list.d/docker.list',
  require => Exec['docker-gpg'],
}
```

```
# Instalar Docker y el plugin de Compose
package { ['docker-ce', 'docker-ce-cli', 'containerd.io',
          'docker-buildx-plugin', 'docker-compose-plugin']:
  ensure => installed,
  require => Exec['docker-repo'],
}

# Asegurar que el servicio Docker esté activo
service { 'docker':
  ensure => running,
  enable => true,
  require => Package['docker-ce'],
}
```

```
1 exec { 'apt-update':
2   command => '/usr/bin/apt-get update',
3 }
4
5 package { ['ca-certificates', 'curl', 'gnupg']:
6   ensure => installed,
7   require => Exec['apt-update'],
8 }
9
10 exec { 'docker-gpg':
11   command => '/bin/bash -c "install -m 0755 -d /etc/apt/keyrings && curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg"',
12   creates => '/etc/apt/sources.list.d/docker.list',
13   require => Package['curl'],
14 }
15
16 exec { 'docker-repo':
17   command => '/bin/bash -c "echo deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu jammy stable > /etc/apt/sources.list.d/docker.list"',
18   creates => '/etc/apt/sources.list.d/docker.list',
19   require => Exec['docker-gpg'],
20 }
21
22 package { ['docker-ce', 'docker-ce-cli', 'containerd.io', 'docker-buildx-plugin', 'docker-compose-plugin']:
23   ensure => installed,
24   require => Exec['docker-repo'],
25 }
26
27 service { 'docker':
28   ensure => running,
29   enable => true,
30   require => Package['docker-ce'],
31 }
```

3.5 Creación automática del servidor

Paso 2. Desde la carpeta fase3/ se ejecutó el comando que crea la máquina virtual, la aprovisiona con Puppet y levanta los contenedores:

```
vagrant up
```

```
Administrador: Símbolo del sistema - vagrant up
16/06/2026 12:11 a. m. 1,096 default.app
1 archivos 1,096 bytes
2 dirs 51,817,103,360 bytes libres

C:\Users\RedFo\OneDrive\Escritorio\fase3\vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Importing base box 'ubuntu/jammy64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
==> default: Setting the name of the VM: fase3.default.1781501125429.75072
Vagrant is currently configured to create VirtualBox synced folders with
the 'SharedFoldersEnableSymlinksCreate' option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:

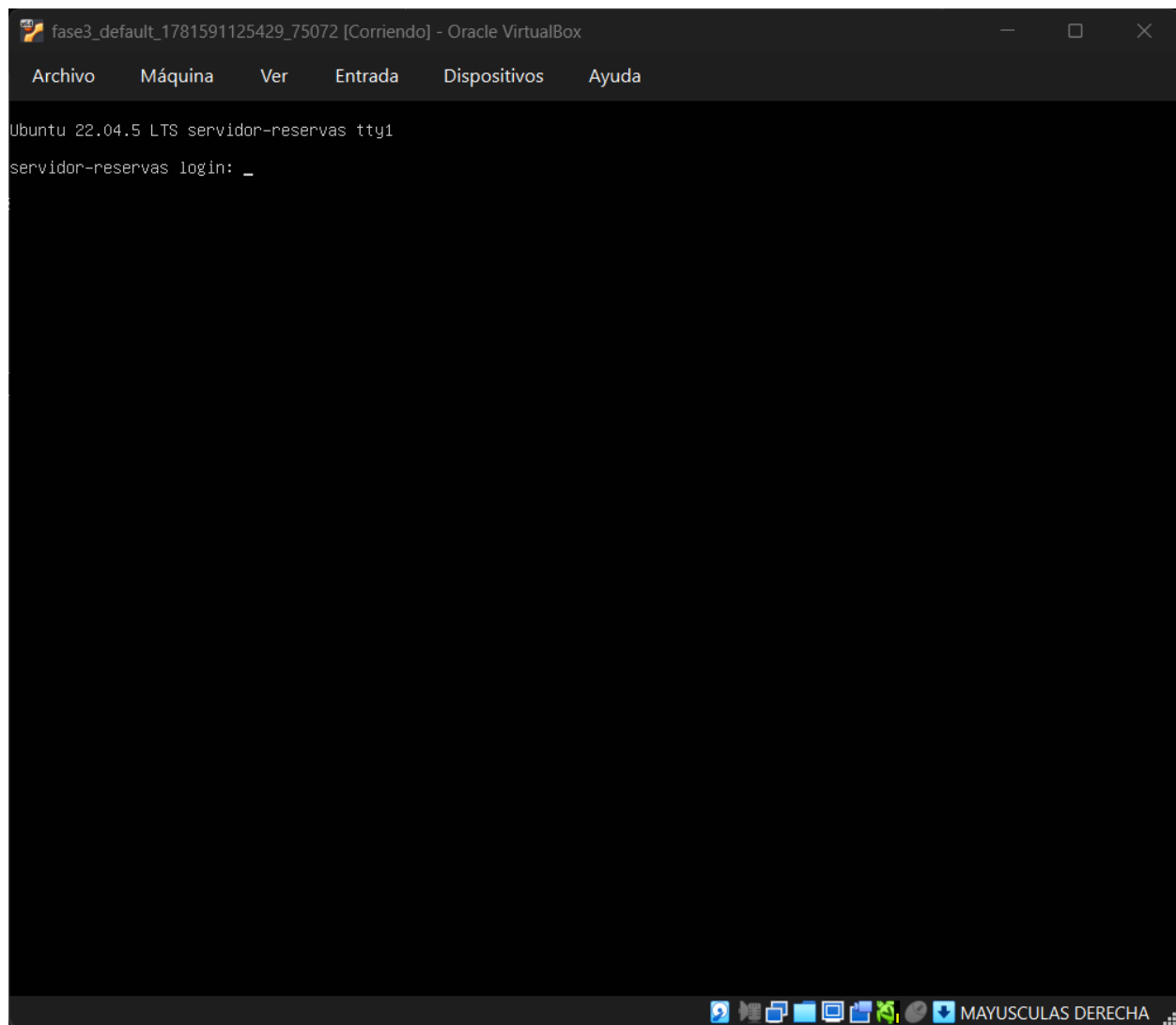
  https://www.virtualbox.org/manual/ch04.html#sharedfolders

This option can be disabled globally with an environment variable:

  VAGRANT_DISABLE_VBOXSYMLINKCREATE=1

or on a per folder basis within the Vagrantfile:

  config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnableSymlinksCreate: false
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
  default: Adapter 1: nat
  default: Adapter 2: hostonly
==> default: Forwarding ports...
  default: 8080 (guest) => 8080 (host) (adapter 1)
  default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
  default: SSH address: 127.0.0.1:2222
  default: SSH username: vagrant
  default: SSH auth method: private key
```



3.6 Verificación dentro del servidor

Paso 3. Se ingresó al servidor recién creado y se comprobó que Docker y los contenedores estuvieran funcionando:

```
vagrant ssh  
  
cd /home/vagrant/app  
sudo docker compose ps
```

```
vagrant@servidor-reservas: / ^ X + v
Processes: 108
Users logged in: 0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:dc:5fff:fe02:c88d

Expanded Security Maintenance for Applications is not enabled.

43 updates can be applied immediately.
35 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

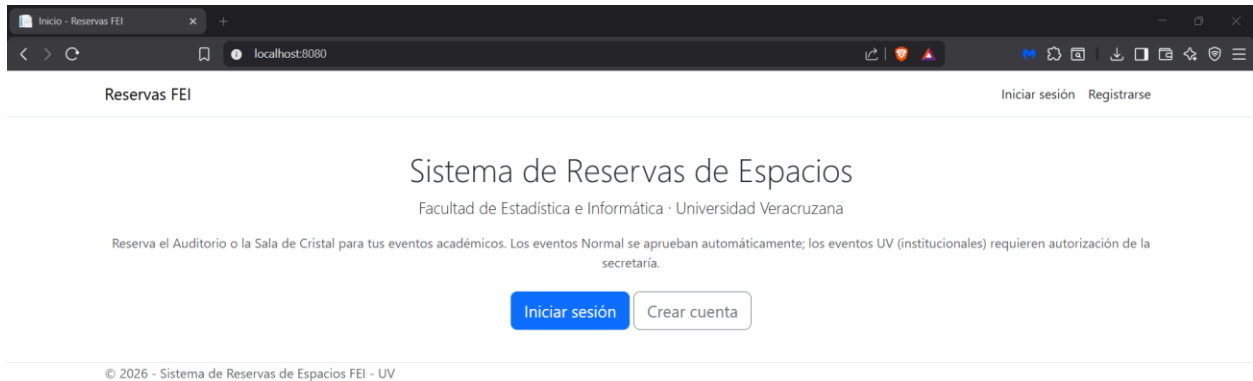
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

vagrant@servidor-reservas:~$
vagrant@servidor-reservas:~$
vagrant@servidor-reservas:~$ docker --version
cd /vagrant_app
docker compose ps
Docker version 29.5.3, build d1c06ef
permission denied while trying to connect to the docker API at unix:///va
r/run/docker.sock
vagrant@servidor-reservas:/vagrant_app$
```

3.7 Verificación del sistema en el navegador

Paso 4. Gracias al reenvío de puertos del Vagrantfile, el sistema se abrió desde el navegador del equipo anfitrión; se ingresó con `admin@fei.uv.mx` / `Admin123!`:

```
http://localhost:8080
# o bien: http://192.168.56.10:8080
```



Resumen de evidencias por fase

Capturas incluidas como evidencia en cada fase:

Fase	
Fase 1	SO instalado, MySQL activo, base «reservas_fei» creada, SDK de .NET, migraciones aplicadas y sistema funcionando en el navegador.
Fase 2	Versiones de Docker, Dockerfile, docker-compose.yml, contenedores en «Up» y sistema funcionando desde los contenedores.
Fase 3	Versión de Vagrant, Vagrantfile, manifiesto de Puppet, salida de «vagrant up», Docker instalado por Puppet y sistema funcionando desde el navegador del anfitrión.

Conclusión

A lo largo de las tres fases se observó una evolución desde un despliegue manual hacia uno totalmente automatizado. En la Fase 1 se instalaron y configuraron a mano MySQL y el SDK de .NET, y se aplicaron las migraciones de Entity Framework Core sobre el servidor: ofrece control total, pero es lento y difícil de reproducir. En la Fase 2, Docker y Docker Compose permitieron empaquetar el sistema ReservaEspacios y su base de datos en contenedores y levantarlos con un solo comando, garantizando un entorno consistente. Finalmente, en la Fase 3, Vagrant y Puppet automatizaron la creación y configuración del servidor, de modo que toda la infraestructura puede recrearse desde cero de forma reproducible. En conjunto, las tres fases mostraron cómo la contenerización y la infraestructura como código reducen errores, aceleran el despliegue y facilitan la portabilidad del aplicativo. Hubo una reducción significativa en la creación de máquinas virtuales.